

BriskSeed: When Is Historical Search-State Reuse Useful?

Advisor Claim-Boundary and Evidence Contract

Advisor-authored narrative; implementation and experiments remain student-owned

August 2026

Abstract

BriskSeed reuses successful approximate-nearest-neighbor (ANN) search outcomes as seeds while an index evolves. The BriskSeed branch contains an extensive student manuscript and result figures, but its current paper does not compile under Tectonic and its numerical claims are not bound to a single result/provenance manifest. This independent draft therefore treats those numbers as branch-local historical reports, not verified results. It isolates the publishable question: under what drift, query recurrence, and memory budget does seed reuse beat the strongest dynamic-ANN backend at matched recall and total update-plus-query cost? We freeze baselines, oracles, evidence gates, and failure conditions. No experiment was run and the student manuscript is unchanged.

1 Seven-Question Storyline

- Q1. Problem.** In a continuously updated ANN graph, when can a prior query’s result set safely and profitably seed later traversal, and when does drift make reuse harmful?
- Q2. Importance.** Dynamic vector services must jointly absorb updates and serve queries. Blind reuse risks recall; rebuilding auxiliary guidance consumes update budget; always falling back loses the opportunity to avoid random graph accesses.
- Q3. Related-work assumption.** HNSW is a strong configurable search baseline [3]; GraphReorder targets cache locality but assumes an expensive layout transformation [1]; Wolverine repairs paths after updates [2]; CANDOR-Bench exposes open-world update/query interference [4]. None of these establishes that historical query outcomes remain useful under drift. BriskSeed must separate its reuse mechanism from the benchmark contribution and from backend update quality.
- Q4. Mechanism hypothesis.** Keep a bounded hot tier for exact/high-confidence recurrent queries and a bounded semantic tier for long-tail candidates. Retrieve a seed, perform limited native graph expansion, validate its boundary/overlap receipt, and fall back to the unmodified backend when confidence fails. The hypothesis is that avoided traversal exceeds lookup, validation, writeback, and stale-seed recovery cost.
- Q5. Feasibility.** The branch contains a two-tier StreamSeed implementation, a plugin seam, runbooks, figures, and CSV summaries. These demonstrate implementation and historical-analysis presence. They do not yet prove a reproducible mechanism result because exact command, exit, hardware, dataset, backend pin, and raw-to-figure linkage are not unified.
- Q6. Evaluation.** Section 3 specifies matched arms, correctness oracle, metrics, provenance, success gate, and stopping conditions before a held-out run.
- Q7. Takeaway.** A positive paper would identify a predictive reuse boundary over drift, recurrence, and memory. A negative paper would show that robust native search/fallback cost eliminates the apparent gain, closing seed reuse for that regime without weakening CANDOR-Bench.

2 Architecture and Predictive Boundary

For query generation g , each seed receipt binds query identity/signature, source generation, returned IDs, reuse kind, age, and subsequent validation decision. Update receipts bind inserted/deleted IDs and the exact backend state. A seed hit is not a success: it succeeds only if the accepted result satisfies the exact top- k oracle and reduces total work. The hypothesis predicts lower gains as unseen-query fraction, update batch size, or distribution shift rises; larger gains when recurrence and top- k stability rise; and a backend-dependent crossover because native update repair changes fallback quality. The branch-reported FreshDiskANN recall decrease is therefore a negative boundary, not noise to tune away.

3 Matched Evaluation Contract

Arms. B0 is the same backend with BriskSeed disabled and its strongest deployable search configuration. B1 caches exact query results with invalidation, separating ordinary caching from seed-guided traversal. B2 periodically rebuilds static guidance with the rebuild interval swept. B3 is an oracle that chooses native versus seeded search after executing both; it measures headroom but is not deployable. Treatment is the frozen two-tier BriskSeed policy. Report HNSW, SymphonyQG, FreshDiskANN, and any dynamic repair backend as separate strata; never pool them as repetitions.

Matching and provenance. Freeze Git/submodule pins, compiler flags, CPU/NUMA identity, dataset and split hashes, distance metric, ordered update/query streams, backend construction/search parameters, threads, seeds, and memory cap. Compare latency- or work-matched Pareto frontiers, not one hand-selected `efSearch`. Charge seed lookup, validation, fallback, writeback, eviction, and memory to the treatment.

Oracle. Recompute exact nearest neighbors against the post-update live set at every measured generation. Require 100% identity conservation: no deleted IDs, duplicates, omissions, cross-generation seed use, or query/ground-truth mismatch. Report $\text{Recall}@k$ from the exact oracle and verify that fallback invokes the identical B0 path/configuration.

Metrics and gate. Report update latency/throughput, query QPS and p95, $\text{Recall}@k$, distance computations, nodes visited, cache misses, fallback rate, seed age/hit/accept rates, bytes, and peak memory with full repeated-run distributions. A positive claim requires a repeated deployable winner switch over the strongest B0–B2 frontier *and* B3 oracle headroom. Recall loss must remain within a student-preregistered bound for every backend, not just on average; numeric margins and repetition counts are TBD until frozen before held-out execution.

Stops. Stop if the gain vanishes after charging maintenance/fallback, relies on repeated identical queries, fails on a second drift family/backend, is matched by exact caching, or violates any identity/recall gate. Do not retune validation thresholds after observing held-out failures.

4 Evidence Ledger and Two-Week Closure

Artifact	Level	Allowed statement
Student manuscript/figures	historical branch report	Motivate hypotheses only; current Tectonic build fails and no unified provenance manifest binds claims.
Summary CSV files	historical summary	Values are not raw results and are not promoted to real-online evidence.
StreamSeed source/plugin	implementation presence	Establishes a mechanism carrier, not superiority.
This PDF	derived artifact	Establishes only reproducible advisor narrative compilation.

Week one: the student owner freezes all identities, links raw rows to figures/claims, repairs the formal manuscript build, and validates generation-aware exact oracles. Week two: the owner runs matched B0–B3/treatment sweeps and applies the frozen gate. Advisor ownership ends at narrative and acceptance criteria; code, environment, execution, debugging, and results remain student-owned. BriskSeed is currently CPU dynamic-ANN work, so an NPU six-link claim is inapplicable; accelerator mapping without a native counterexample and mechanism is adaptation.

5 Conclusion

The credible contribution is not “reuse is always faster,” but a falsifiable boundary for when historical state remains useful after updates. The contract protects that boundary from uncharged maintenance, weak baselines, and unbound historical plots.

References

- [1] Benjamin Coleman, Santiago Segarra, Alexander J. Smola, and Anshumali Shrivastava. Graph reordering for cache-efficient near neighbor search. In *NeurIPS*, volume 35, pages 38488–38500, 2022.
- [2] Dawei Liu, Bolong Zheng, Ziyang Yue, Fuhao Ruan, Xiaofang Zhou, and Christian S. Jensen. Wolverine: Highly efficient monotonic search path repair for graph-based ann index updates. *PVLDB*, 18(7):2268–2280, 2025.
- [3] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4):824–836, 2020.
- [4] Mingqi Wang, Junyao Dong, Zhuoyan Wu, et al. CANDOR-Bench: Benchmarking in-memory continuous ANNS under dynamic open-world streams. *Proceedings of the ACM on Management of Data*, 4(1), 2026.